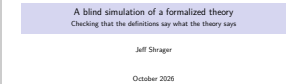
# A blind simulation of a formalized theory

## Checking that the definitions say what the theory says

Jeff Shrager

October 2026

# A blind simulation of a formalized theory




- A methods talk. The question is one every formal model faces, and the procedure is usable without Lean.
- The materials are in `MultiDigit/sim/` in the repository: the specification, the preregistration, the delivered simulator, the harness, and the results.

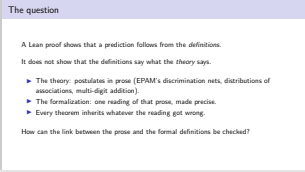
# The question

A Lean proof shows that a prediction follows from the *definitions*.

It does not show that the definitions say what the *theory* says.

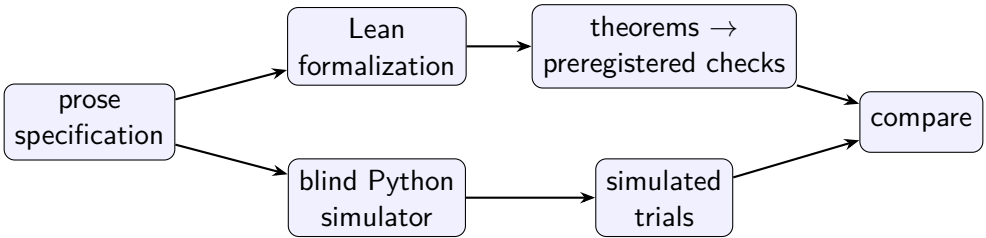
- The theory: postulates in prose (EPAM's discrimination nets, distributions of associations, multi-digit addition).
- The formalization: one reading of that prose, made precise.
- Every theorem inherits whatever the reading got wrong.

How can the link between the prose and the formal definitions be checked?




- This is the first objection a skeptical audience raises, and it is fair: machine-checked proofs move the risk from the derivation to the definitions.
- Simulation papers have the same problem in another form: the published description and the code can differ, and readers see only the description.

# The idea: two independent readings

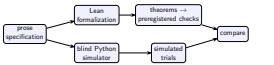



N-version programming (Avižienis 1985), applied to a theory. A disagreement means one of three things: the prose is ambiguous, the formalization drifted, or the simulator has a bug.

# A blind simulation of a formalized theory

## The idea: two independent readings

The idea: two independent readings




N-version programming (Avižienis 1985), applied to a theory. A disagreement means one of three things: the prose is ambiguous, the formalization drifted, or the simulator has a bug.

- In fault-tolerant software, several teams implement one specification independently, and their disagreements locate the faults. Here the two versions are the Lean formalization and a simulator written without seeing it.
- The Lean side also exports exact outcome distributions for shared instances, so some comparisons are exact, not just statistical.

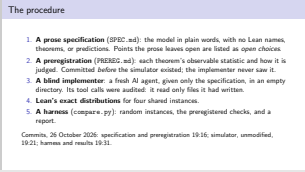
# The procedure

1. **A prose specification** (`SPEC.md`): the model in plain words, with no Lean names, theorems, or predictions. Points the prose leaves open are listed as *open choices*.
2. **A preregistration** (`PREREG.md`): each theorem's observable statistic and how it is judged. Committed *before* the simulator existed; the implementer never saw it.
3. **A blind implementer**: a fresh AI agent, given only the specification, in an empty directory. Its tool calls were audited: it read only files it had written.
4. **Lean's exact distributions** for four shared instances.
5. **A harness** (`compare.py`): random instances, the preregistered checks, and a report.

Commits, 26 October 2026: specification and preregistration 19:16; simulator, unmodified, 19:21; harness and results 19:31.

# A blind simulation of a formalized theory

## The procedure




- The commit timestamps are the preregistration's evidence: the checks were fixed before any simulator output existed.
- The simulator is committed exactly as delivered. Disagreements are recorded, not fixed in place.

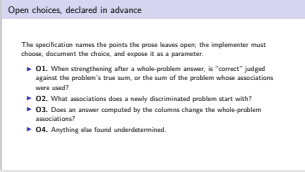
# Open choices, declared in advance

The specification names the points the prose leaves open; the implementer must choose, document the choice, and expose it as a parameter.

- **O1.** When strengthening after a whole-problem answer, is "correct" judged against the problem's true sum, or the sum of the problem whose associations were used?
- **O2.** What associations does a newly discriminated problem start with?
- **O3.** Does an answer computed by the columns change the whole-problem associations?
- **O4.** Anything else found underdetermined.

# A blind simulation of a formalized theory

## Open choices, declared in advance




- Declaring open choices separates "the theory doesn't say" from "the implementer got it wrong". Where the theory is silent, every reasonable choice is legitimate, and the comparison can test whether the predictions depend on it.
- O2 turned out to matter a great deal.

# How each check is judged

- **Exact relations**: zero violations allowed, over every trial whose hypothesis holds. For example, an error is the confused neighbour's answer.
- **Probabilities**: pass if $|\hat{p} - p| < 4\sqrt{p(1-p)/N}$. For example, column use $= (1-R)^n$.
- **No-change and monotone relations**: for example, learning other problems leaves a practiced problem's trial unchanged.
- **Non-vacuity**: each check reports how many trials met its hypothesis. A check whose hypothesis never held is reported as *vacuous*, not as passed.

Instances: 4 Lean-exact; 30 random with learning off; 20 random with learning on, across the open choices.

# A blind simulation of a formalized theory

## How each check is judged

- The non-vacuity rule is easy to forget and essential: a theorem with an unmet hypothesis "passes" every test trivially.

How each check is judged

- **Exact relations**: zero violations allowed, over every trial whose hypothesis holds. For example, an error is the confused neighbour's answer.
- **Probabilities**: pass if $|\hat{p} - p| < 4\sqrt{p(1-p)/N}$. For example, column use $= (1 - R)^a$.
- **No-change and monotone relations**: for example, learning other problems leaves a practiced problem's trial unchanged.
- **Non-vacuity**: each check reports how many trials met its hypothesis. A check whose hypothesis never held is reported as vacuous, not as passed.

Instances: 4 Lean-exact; 30 random with learning off; 20 random with learning on, across the open choices.

# Results: no failures

| Check | Lean theorem | kind | tests | failures |
|---|---|---|---|---|
| Lean exact | `#eval MD.trial` | prob | 243 | 0 |
| P1 | `columns_prob` | prob | 394 | 0 |
| P2 | `whole_depends_only_on_route` | prob | 8543 | 0 |
| P3 | `confusion_gives_neighbours_answer` | exact | 200 | 0 |
| P4 | `confident_errors` | exact+prob | 222 | 0 |
| P7 | `learning_preserves_trial` | exact | 74692 | 0 |
| P9 | `columns_place_value` | exact | 248800 | 0 |
| P11 | `discrimination_removes_confusion` | exact | 91090 | 0 |
| P6a, P6b, P14, P17, P18 | | | 0 | *vacuous* |

The prose and the formalization agree on recognition, EPAM learning, retrieval, both backups, and carrying.

# A blind simulation of a formalized theory

## Results: no failures

Results: no failures

| Check | Lean theorem | kind | tests | failures |
|---|---|---|---|---|
| Lean exact | #eval RD.trial | prob | 243 | 0 |
| P1 | columns_pcnh | prob | 394 | 0 |
| P2 | whole_depends_only_on_route | prob | 8543 | 0 |
| P3 | confusion_gives_neighbours_answer | exact | 200 | 0 |
| P4 | confident_errors | exact+prob | 222 | 0 |
| P7 | learning_preserves_trial | exact | 74602 | 0 |
| P9 | columns_place_value | exact | 248800 | 0 |
| P10 | discrimination_removes_confusion | exact | 91090 | 0 |
| P6a, P6b, P14, P17, P18 | | | 0 | vacuous |

The prose and the formalization agree on recognition, EPAM learning, retrieval, both backups, and carrying.

- A selection of rows; the full table (22 checks) is in `MultiDigit/sim/RESULTS.md`.
- "Lean exact": 243 outcome frequencies on the four shared instances, each within tolerance of Lean's exact probability.

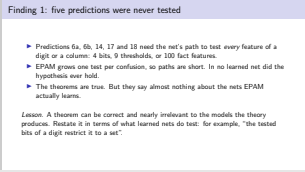
# Finding 1: five predictions were never tested

- Predictions 6a, 6b, 14, 17 and 18 need the net's path to test *every* feature of a digit or a column: 4 bits, 9 thresholds, or 100 fact features.
- EPAM grows one test per confusion, so paths are short. In no learned net did the hypothesis ever hold.
- The theorems are true. But they say almost nothing about the nets EPAM actually learns.

*Lesson.* A theorem can be correct and nearly irrelevant to the models the theory produces. Restate it in terms of what learned nets do test: for example, "the tested bits of a digit restrict it to a set".

# A blind simulation of a formalized theory

## Finding 1: five predictions were never tested




- Without the non-vacuity rule, these would have been reported as five passes.
- This is a finding about proving things in cognitive science generally: hypotheses that make a proof go through can quietly exclude the cases of interest.

# Finding 2: a gap in the formalization

- In Lean, sampling divides by total strength. With total strength 0, a trial has *no* outcome at all: probability mass 0.
- The theorems avoid this by assuming total strength $\neq 0$.
- The simulator read it as "retrieval exhausted, go to the columns". That is the natural reading of the model.
- It matters: a newly discriminated problem with no associations (the most natural choice of O2) is exactly this case, and it lies outside every theorem.

A gap in the formalization, not a simulator bug.

Finding 2: a gap in the formalization

- In Lean, sampling divides by total strength. With total strength 0, a trial has *no* outcome at all: probability mass 0.
- The theorems avoid this by assuming total strength $\neq 0$.
- The simulator read it as "retrieval exhausted, go to the columns". That is the natural reading of the model.
- It matters: a newly discriminated problem with no associations (the most natural choice of O2) is exactly this case, and it lies outside every theorem.

A gap in the formalization, not a simulator bug.

- The fix is to define search with total strength 0 as exhausted. It is recorded as an open issue.
- The proofs' hypothesis `total` $\neq$ `0` was visible all along; the simulation showed that it excludes a case the model needs.

# Finding 3: a prediction depends on an open choice

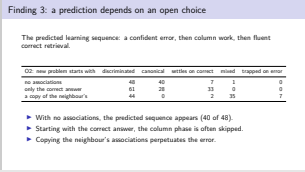
The predicted learning sequence: a confident error, then column work, then fluent correct retrieval.

| O2: new problem starts with | discriminated | canonical | settles on correct | mixed | trapped on error |
|---|---|---|---|---|---|
| no associations | 48 | 40 | 7 | 1 | 0 |
| only the correct answer | 61 | 28 | 33 | 0 | 0 |
| a copy of the neighbour's | 44 | 0 | 2 | 35 | 7 |

- With no associations, the predicted sequence appears (40 of 48).
- Starting with the correct answer, the column phase is often skipped.
- Copying the neighbour's associations perpetuates the error.

# A blind simulation of a formalized theory

## Finding 3: a prediction depends on an open choice




- This part was exploratory, not preregistered: learning over many trials is not yet proved in Lean.
- The model leaves O2 open, and the prediction depends on it. Which reading is right is a substantive question about the theory, now sharply posed.

# Finding 4: ambiguities resolved the same way, blind

The implementer met ambiguities the specification did not flag, and resolved them as the Lean formalization had:

- single-digit associations belong to *ordered* pairs ($3 + 5$ and $5 + 3$ separately);
- whole-problem strengthening applies to the associations that were actually retrieved from.

Agreement found independently is evidence that these are the natural readings, and it should have been in the specification.

# A blind simulation of a formalized theory

## Finding 4: ambiguities resolved the same way, blind

Finding 4: ambiguities resolved the same way, blind

The implementer met ambiguities the specification did not flag, and resolved them as the Lean formalization had:

- single-digit associations belong to *ordered* pairs ($3 \div 5$ and $5 \div 3$ separately);
- whole-problem strengthening applies to the associations that were actually retrieved from.

Agreement found independently is evidence that these are the natural readings, and it should have been in the specification.

- The implementer's notes (`IMPLEMENTATION_NOTES.md`) list every choice it made; these two were compared with the Lean definitions after the fact.

# Limits

- **Blindness is procedural.** It rests on instructions and an audit of tool calls, not on anything that prevents reading.
- **Not fully independent.** The implementer was a model of the same family as the one that wrote the specification and the formalization, so shared misreadings are possible. A human implementer, or community code, would be stronger.
- **One model, one round.** It checked the EPAM + DOA multi-digit model, before memory became a plug-in and before the ACT-R version existed.
- **Simulation covers instances.** It can expose a mismatch; it cannot certify the definitions in general.

# A blind simulation of a formalized theory

## Limits

Limits

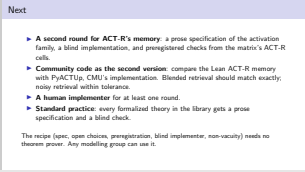
- **Blindness is procedural.** It rests on instructions and an audit of tool calls, not on anything that prevents reading.
- **Not fully independent.** The implementer was a model of the same family as the one that wrote the specification and the formalization, so shared misreadings are possible. A human implementer, or community code, would be stronger.
- **One model, one round.** It checked the EPAM + DOA multi-digit model, before memory became a plug-in and before the ACT-R version existed.
- **Simulation covers instances.** It can expose a mismatch; it cannot certify the definitions in general.

- The procedure's strength is that its weak points are explicit: anyone can see what was blind, what was preregistered, and when.

# Next

- **A second round for ACT-R's memory**: a prose specification of the activation family, a blind implementation, and preregistered checks from the matrix's ACT-R cells.
- **Community code as the second version**: compare the Lean ACT-R memory with PyACTUp, CMU's implementation. Blended retrieval should match exactly; noisy retrieval within tolerance.
- **A human implementer** for at least one round.
- **Standard practice**: every formalized theory in the library gets a prose specification and a blind check.

The recipe (spec, open choices, preregistration, blind implementer, non-vacuity) needs no theorem prover. Any modelling group can use it.




- PyACTUp's retrieval samples logistic noise and takes the most active chunk; the Lean memory uses the Boltzmann (softmax) form, which approximates it. Blending uses the Boltzmann weights directly, so that comparison can be exact.